Spark Join Strategies

Broadcast hash Sort merge Shuffled hash Nested loop

Learning objective. Understand how Spark physically executes a join, recognize the strategy in an execution plan, and choose an optimization that matches the data rather than forcing a hint blindly.

Core idea. The logical join type determines the rows returned, while the physical join strategy determines how Spark computes those rows and therefore its cost.

Logical type and physical strategy

Level	Examples	Question answered
Logical join	inner, left, right, full, semi, anti, cross	Which rows must appear in the result?
Physical strategy	broadcast hash, sort merge, shuffled hash, nested loop	How should Spark calculate the result?

1 Broadcast Hash Join

Principle. Spark sends the complete small relation to every executor. Each executor builds an in-memory hash table and probes it while scanning its partitions of the large relation.

How Spark executes it

1. Select the small side of the join.
2. Broadcast that relation to every executor.
3. Build a local hash table keyed by the join column.
4. Probe the hash table while reading the large table locally.

Scala example

```
import org.apache.spark.sql.functions.broadcast

val result = orders.join(
  broadcast(countries),
  Seq("country_code"),
  "inner"
)
```

Advantages	Risks and limitations
• Avoids shuffling the large table • Usually the fastest option for a small dimension table • Simple equality lookup in memory	• The broadcast side must fit in each executor's memory • Broadcasting an underestimated table can cause memory pressure • Not suited to two large inputs

Typical use case. A large orders table joined with a country or product category reference table containing only a few hundred or thousand rows.

2 Sort Merge Join

Principle. Spark repartitions both relations on the join key, sorts the partitions, and then walks through the two sorted streams to match equal keys.

How Spark executes it

1. Shuffle both sides by the join key unless partitioning already matches.
2. Sort each resulting partition by the key.
3. Merge the two sorted streams and emit matching rows.

Scala example

```
val result = orders.join(
  customers,
  Seq("customer_id"),
  "inner"
)
```

Advantages	Risks and limitations
• Designed for two large data sets • Does not require one complete side to fit in memory • A robust default for large equi-joins	• Shuffle transfers data across the network • Sorting consumes CPU and can spill to disk • Hot keys may create skewed, slow partitions

Typical use case. Two large fact or transaction tables joined using an equality condition such as orders.customer_id = customers.customer_id.

3 Shuffled Hash Join

Principle. Spark shuffles both relations by the join key, then builds a hash table from the smaller side inside each partition. Unlike a broadcast join, no executor receives the complete small table.

How Spark executes it

1. Shuffle both relations so equal keys reach the same partition.
2. Choose the smaller side within each matching partition.
3. Build a partition-local hash table.
4. Probe it with rows from the larger partition.

Scala example

```
val result = orders.join(
  customers.hint("SHUFFLE_HASH"),
  Seq("customer_id"),
  "inner"
)
```

Advantages	Risks and limitations
• Avoids the sorting phase of a sort merge join • Useful when one side is clearly smaller after partitioning • Can outperform sort merge when partitions fit comfortably in memory	• Both sides normally shuffle • The build side of each partition must fit in memory • Skew can make one hash table much larger than the others • A hint is a preference, not an absolute guarantee

Typical use case. One input is too large to broadcast globally but is still much smaller than the other input after repartitioning.

4 Nested Loop Join

Principle. Spark tests candidate row pairs because the condition cannot be handled as a simple hash or sort merge equality lookup. In the worst case, work grows approximately with the product of the two input sizes.

How Spark executes it

1. Choose a build side when broadcasting is possible.
2. For each row on the streamed side, evaluate the condition against candidate rows on the other side.
3. Return the pairs for which the condition is true.

Scala example

```
val result = orders.join(
  broadcast(discountRules),
  orders("amount").between(
    discountRules("minimum"),
    discountRules("maximum")
  )
)
```

Advantages	Risks and limitations
• Supports non-equality conditions such as <, > and between • Broadcast nested loop can be acceptable when one side is genuinely tiny • Works for cases that hash joins cannot express	• Potential cost is O(n × m) • Can create a huge number of comparisons or output rows • Dangerous when both sides are large • Cross joins require especially careful validation

Typical use case. Range matching, spatial-like predicates, or another non-equality condition with a very small rule table.

Comparison at a glance

Strategy	Typical inputs	Shuffle	Sort	Main memory need	Best fit
Broadcast hash	Large + small	Large side no	No	Small table per executor	Small dimension table
Sort merge	Large + large	Usually both	Yes	Moderate with possible spill	Large equi-join
Shuffled hash	Large + smaller	Usually both	No	Build side per partition	Unequal sizes after shuffle
Nested loop	Any + tiny preferred	Varies	No	Varies	Non-equality condition

How to identify the selected strategy

Ask Spark to display the physical plan:

```
result.explain("formatted")
```

Look for one of these operators:

- BroadcastHashJoin
- SortMergeJoin
- ShuffledHashJoin
- BroadcastNestedLoopJoin
- CartesianProduct

Practical decision guide

Is one side small enough to replicate safely? Prefer Broadcast Hash Join.

Are both sides large and joined by equality? Sort Merge Join is usually the safe default.

Is one side much smaller after shuffle and each build partition fits in memory? Consider Shuffled Hash Join.

Is the condition non-equivalent, for example a range? A Nested Loop strategy may be necessary; keep one side tiny.

Optimization checklist

- Inspect the physical plan before and after an optimization.
- Keep table statistics current so Spark can estimate input sizes.
- Broadcast only a relation that safely fits in every executor's memory.
- Project only useful columns before the join to reduce network and memory costs.
- Filter as early as possible to reduce the number of rows entering the join.
- Watch for skewed keys, especially a single customer or user identifier with many more rows than the others.
- Use hints only when measurement shows that Spark's chosen plan is unsuitable.
- Compare runtime, shuffle read and write, spill, task duration and partition imbalance in the Spark UI.

Mini exercise

For each scenario, choose the most likely strategy and justify the choice.

1. Two billion orders joined with a table of 200 country codes.

2. Two transaction tables, each several hundred gigabytes, joined on customer_id.

3. A large event table matched against 30 time-range rules using between.

4. A 200 GB table joined with a 2 GB table that cannot be broadcast, but whose build partitions fit in memory.

Suggested answers

1. Broadcast Hash Join: the country table is a small dimension table.

2. Sort Merge Join: both inputs are large and the condition is an equality.

3. Broadcast Nested Loop Join: the range predicate is not a simple equality and the rule table is tiny.

4. Shuffled Hash Join: the smaller side is too large to replicate globally but can be hashed safely per partition.

Key takeaway

The logical join defines correctness. The physical join strategy defines cost. A good Spark optimization starts with the execution plan and data sizes, then validates the choice with metrics rather than assumptions.